

#	LECTURES NAME	MODULES (NO)
1	Introduction to Computer Science	1-28 (28)
2	Data Compression:	29-31 (3)
3	Data Manipulation:	32-44 (13)
4	Operating Systems:	45-58 (14)
5	Networking and the Internet:	59-84 (26)
6	Algorithm:	85-108 (24)
7	Programming Languages	109-125 (17)
8	Software Engineering:	126-145 (20)
9	Data Abstraction:	146-149 (4)
10	Database Systems:	150-169 (20)
11	Artificial Intelligence:	170-175 (6)
12	CS Impact:	176-186 (11)
13	Word Processing:	187-214 (28)
14	Presentations:	215-219 (5)
15	Spreadsheets:	220-224 (5)
16	Database:	225-229 (5)
17	Web page Development:	230-234 (5)

Introduction to Computer Science – CS101

Lecture 1 – Introduction to Computer Science

Modules (1 to 28)

What is Computer Science?

Computer science (CS) is the study of computers and algorithmic processes, including their principles, their hardware and software designs, their applications, and their impact on society. Computer Science is the discipline that seeks to build a scientific foundation for such topics as:

- Hardware
- Software
- Programming
- Networks
- Graphics
- Robots
- Database
- Security
- Algorithmic Solutions
- Information Processing

Computer Science Applications:

- Telecom
- Banks
- Hospitals
- Software Development
- Service Industry
- Pak Army
- Freelancing
- and many more

Local and International Job Market:

- Artificial Intelligence and Machine Learning
- Data Science
- Virtual Reality
- Back-End Developer
- Front-End Developer
- UI Designer
- Full-Stack Engineer
- IT Manager

Are you not a student of Computer Science?

Breadth First Learning: This course will give basic introduction to almost all courses of Computer Science. Such strategy of learning is known as **Breadth First Learning**. The other strategy is called **Depth First Learning** in which one particular course is first covered in details and then next course is covered. The Breadth First Learning helps students to first know what they will be learning in the whole degree program.

We will cover:

1. Abstract view of all major courses in CS
2. Understanding what will be studied in CS
3. Why each course is important ?
4. Clarifying the bigger Picture

Introduction to the topics will be covered in this course:

1. Search Engine Usage Techniques
2. History of Computing
3. Data Storage
4. Data Manipulation
5. Operating System
6. Networking and the Internet
7. Algorithms
8. Programming Languages
9. Software Engineering
10. Data Abstraction
11. Database Systems
12. Artificial Intelligence
13. CS impact on society
14. Content Filtering, Spam, International law
15. Word Processing
16. Presentations Development
17. Spreadsheet
18. Database MS Access
19. Web Page Development

Search Engines:

Search Engine like Google index web pages and helps you to retrieve the relevant web pages based on your queries. Starting from today, in next few modules, we will focus on techniques and short-cuts to retrieve most relevant information from Google.

1. Searching Tricks :

- a. Weather Searching
 - b. Performing Calculations
 - c. Currency Conversion
2. Search Operators
 - a. Search on Social Media(@)
 - b. Search for a price(pkr)
 - c. Searching Hash tags(#)
 - d. Exclude words from Query(-)
 - e. Exact Match(" ")
 - f. Wild Card based Searching(*)
 - g. Boolean Operators(AND, OR)
 - h. Search with a Specific Site("virtual university site: youtube.com")
 - i. Searching Related Websites("related:youtube.com")
 - j. Searching a cached version("cache:youtube.com")
 - k. File type("CS101 Handouts filetype: pdf")
 - l. Define Operator("Define: Computer")
 - m. Tilt(Rotate the Google screen to 360 degree)
3. Advanced Search Operators
 - a. Intitle
 - b. Allintitle
 - c. inurl and allinurl
 - d. Intext and allintext
 - e. Proximity Search(Education AROUND(3) "virtual university")
 - f. Let's solve a complex Query(site:youtube.com –inurl:https)

What we should not search on Internet:

There are three main reasons:

1. Google Adds
2. Security Agencies
3. Cyber-security attacks

Roots of Computing:

1. Abacus
2. Gears
3. Punch Cards
4. ENIAC (Electronic Numerical Integrator And Calculator) developed by John Mauchly and J. Presper Eckert at the Moore School of Electrical Engineering, University of Pennsylvania.

Factsheet of ENIAC:

- Occupied 1800 square feet

- 20, 000 vacuum tubes
- 1500 relays
- 10, 000 capacitors
- 70, 000 registers
- 200 Kilo Watt electricity
- Weight 30tons
- Cost = \$487, 000
- PKR = 62.5 millions

5. Rapid Advancement

Bits in Computer, all kind of information is stored in bits. Bit is the basic unit of storage.

Basics:

- Information is coded as pattern of 0 or 1
- Short form of Binary Digits
- One bit can contain only one value 0 or 1

What Bits can represent:

- Representing numbers, text, audio, video, image etc.
- In Chip electric Charge 0/1

Bits units:

Unit	Equivalent
1 Byte	8 bits
1 Kilo Byte (KB)	1024 Bytes
1 Mega Byte (MB)	1024 KB
1 Giga Byte (GB)	1024 MB
1 Tera Byte (TB)	1024 GB

Boolean Operations:

- AND (It produces the output as 1 when both of the inputs are 1. Otherwise gives 0)
- OR (It produces output of 1 when any of the inputs are 1, Otherwise gives 0)
- XOR (It produces the output of 1 when both inputs are different. It produces 0 when both inputs are same)
- Not Operation (Not operation takes one input, produces 1 when input is 0 and produces 0, when the input is 1)

Hexadecimal Notation: When considering the internal activities of a computer, we must deal with patterns of bits, which we will refer to as a string of bits, some of which can be quite long. A long string of bits is often called a **stream**. Unfortunately, streams are difficult

for the human mind to comprehend. Merely transcribing the pattern 101101010011 is tedious and error prone. To simplify the representation of such bit patterns, therefore, we usually use a shorthand notation called **hexadecimal notation**.

Main Memory: For the purpose of storing data, a computer contains a large collection of circuits (such as flip-flops), each capable of storing a single bit. This bit reservoir is known as the machine's main memory. Main memory is often called random access memory (RAM).

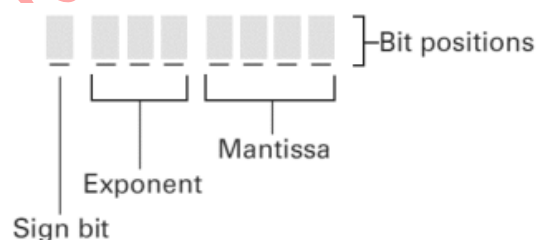
Mass Storage: Due to the volatility and limited size of a computer's main memory, most computers have additional memory devices called mass storage (or secondary storage) systems.

1. **Magnetic System**(Magnetic Disks)
2. **Optical System**(Compact Disk (CD), Digital Versatile Disks (DVDs))
3. **Flash Drives**(SSDs (Solid State Disks), SDs (Secure Digital Memory Cards))

Binary Addition: As we are dealing with two bits while adding them, there could be 4 possibilities

0	1	0	1
+0	+0	+1	+1
0	1	1	10

Floating Point Notation: In contrast to the storage of integers, the storage of a value with a fractional part requires that we store not only the pattern of 0s and 1s representing its binary representation but also the position of the radix point. A popular way of doing this is based on scientific notation and is called floating-point notation. Floating-point notation component given in figure:



Introduction to Computer Science – CS101

Lecture 2 – Data Compression: Modules (29 to 31)

Generic Techniques:

Data Compression: Data compression schemes fall into two categories. Some are lossless, others are lossy. **Lossless** schemes are those that do not lose information in the compression process. **Lossy** schemes are those that may lead to the loss of information

In cases where the data being compressed consist of long sequences of the same value, the compression technique called **run-length encoding**, which is a lossless method, is popular.

Another lossless data compression technique is **frequency-dependent** encoding; a system in which the length of the bit pattern used to represent a data item is inversely related to the frequency of the item's use.

In some cases, the stream of data to be compressed consists of units, each of which differs only slightly from the preceding one. An example would be consecutive frames of a motion picture. In these cases, techniques using relative encoding, also known as **differential encoding**, are helpful.

A variation of dictionary encoding is **adaptive dictionary** encoding (also known as **dynamic dictionary encoding**). In an adaptive dictionary encoding system, the dictionary is allowed to change during the encoding process

Compressing Images:

Another popular compression system for images is **JPEG** (pronounced "JAY-peg"). It is a standard developed by the Joint Photographic Experts Group (hence the standard's name) within ISO. JPEG has proved to be an effective standard for compressing color photographs and is widely used in the photography industry, as witnessed by the fact that most digital cameras use JPEG as their default compression technique.

Still another data compression system associated with images is **TIFF** (short for Tagged Image File Format). However, the most popular use of TIFF is not as a means of data compression but instead as a standardized format for storing photographs along with related information such as date, time, and camera settings. In this context, the image itself is normally stored as red, green, and blue pixel components without compression.

Compressing Audio and Videos:

The most commonly used standards for encoding and compressing audio and video were developed by the **Motion Picture Experts Group (MPEG)** under the leadership of ISO. In turn, these standards themselves are called **MPEG**.

The best-known system for compressing audio is **MP3**, which was developed within the MPEG standards. In fact, the acronym MP3 is short for MPEG layer 3. Among other compression techniques, MP3 takes advantage of the properties of the human ear, removing those details that the human ear cannot perceive. One such property, called **temporal masking**, is that for a short period after a loud sound, the human ear cannot detect softer sounds that would otherwise be audible. Another, called **frequency masking** is that a sound at one frequency tends to mask softer sounds at nearby frequencies.

Introduction to Computer Science – CS101

Lecture 3 – Data Manipulation:

Modules (32 to 44)

CPU (Central Processing Unit): A CPU consists of three parts the **arithmetic/logic** unit, which contains the circuitry that performs operations on data (such as addition and subtraction); the **control unit**, which contains the circuitry for coordinating the machine's activities; and the **register unit**, which contains data storage cells called **registers**, that are used for temporary storage of information within the CPU. Some of the registers within the register unit are considered **general-purpose registers**, whereas others are **special-purpose registers**.

The idea of storing a computer's program in its main memory is called the **stored-program concept**. To apply the stored program concept, CPUs are designed to recognize instructions encoded as bit patterns. This collection of instructions along with the encoding system is called the **machine language**. An instruction expressed in this language is called a **machine level instruction** or, more commonly, a **machine instruction**.

The degree to which machine designs should take advantage of this fact has led to **two philosophies of CPU architecture**. One is that a CPU should be designed to execute a minimal set of machine instructions. This approach leads to what is called a **reduced instruction set computer (RISC)**. The argument in favor of RISC architecture is that such a machine is efficient, fast, and less expensive to manufacture. **Complex instruction set computer (CISC):** The argument in favor of CISC architecture is that the more complex CPU can better cope with the ever-increasing complexities of today's software.

Machine's instructions can be categorized into three groupings:

1. **Data Transfer Group**

The data transfer group consists of instructions that request the movement of data from one location to another. Instructions handle the input/output (I/O) activities of the machine; they are called **I/O instructions** and are sometimes considered as a category in their own right.

2. **Arithmetic/Logic Group**

The arithmetic/logic group consists of the instructions that tell the control unit to request an activity within the arithmetic/logic unit.

3. **Control Group**

The control group consists of those instructions that direct the execution of the program rather than the manipulation of data.

Program Execution:

1. Machine Instruction is fetched from main memory to CPU.
2. Each instruction is decoded and obeyed.
3. The order is as the instructions are stored in memory otherwise specified by a JUMP.

Special Purpose Registers:

- **Instruction Register:** Holding instruction being executed now.
- **Program Counter:** contains the address of next instruction to be executed

Logic Operators (AND, OR, XOR): Bitwise operations that combine two strings of bits to produce a single output string by applying the basic operation to individual columns. This use of the AND operation is an example of the process called **masking**. Here one operand, called a **mask**, determines which part of the other operand will affect the result.

Multiplication:

- Multiplication is repetitive addition
- For example, 8 multiplied by 3, we will add 8 three times

Division:

- Can be achieved through subtraction
- Some small CPUs are designed to have just add, or just add and subtract to do all arithmetic operations.

Role of Controller: Communication between a computer and other devices is normally handled through an intermediary apparatus known as a **controller**. Recently, steps have been taken within the personal computer arena to develop standards, such as the **universal serial bus (USB)** and **FireWire**, by which a single controller is able to handle a variety of devices.

Direct Memory Access and Handshaking: Since a controller is attached to a computer's bus, it can carry on its own communication with main memory during those nanoseconds in which the CPU is not using the bus. This ability of a controller to access main memory is known as **direct memory access (DMA)**, and it is a significant asset to a computer's performance.

Communication media and communication rates:

Communication between computing devices is handled over two types of paths:

1. Parallel communication

- Several signals are transferred at the same time, each on a separate line.
- Data transfer is good
- Requires complex architecture

2. Serial communication

- Transfer data one after the other.
- Requires a simple data path.
- Data transfer rate is relatively slower than parallel communication.
- Ethernet connections for slightly longer distances.
- Rate at which bits are transferred from one computing component to another measured in bits per second (bps)

Pipelining:

Electric pulses travel through a wire no faster than the speed of light. Computer's throughput can be increased without requiring an increase in execution speed involves **pipelining**, which is the technique of allowing the steps in the machine cycle to overlap. In particular, while one instruction is being executed, the next instruction can be fetched, which means that more than one instruction can be in "the pipe" at any one time, each at a different stage of being processed.

www.courses.brain-logix.com

Introduction to Computer Science – CS101

Lecture 4 – Operating Systems: Modules (45 to 58)

Batch processing: the execution of jobs by collecting them in a single batch, then executing them without further interaction with the user.

New operating systems were developed that allowed a program being executed to carry on a dialogue with the user through remote terminals, a feature known as **interactive processing**.

Basic Concepts

Coordination with User:

- Successfully interactive processing is to respond the users with sufficiently fast time.
- Printing record of all students VU versus word processing (typing characters)
- Execution of tasks under a deadline

Real-time Processing:

- Computer performs the tasks in accordance with the deadlines in its external real-world environment.
- Example of Cruise Missile, Radar etc

Interactive system and Real-time Processing:

- If a system is servicing only one user, real-time processing was relatively easier to implement, however, computers in 60s, 70s were expensive,
- Each machine had to serve more than one user at remote terminals.

Time Sharing:

- Based on this problem, such OS were designed to service multiple users at the same time called time-sharing. Active, Ready, waiting states

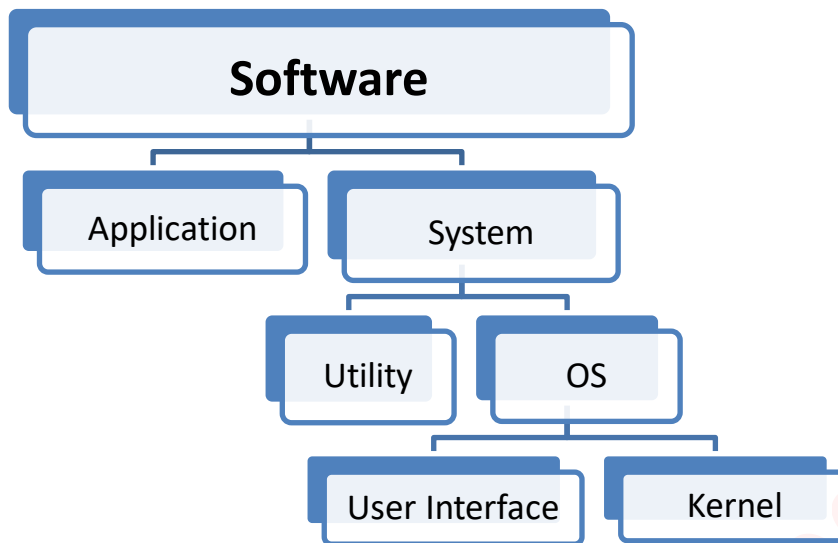
Multi-programming:

- One way of implementing time-sharing, small time intervals, and each job is executed for such a small interval.
- Early systems were able to service 30 users simultaneously.

Multitasking:

- One user executing several tasks simultaneously.
- Today Multiprogramming is used in single user and multiple users.

Software Classification:



Components:

User Interface: In order to perform the actions requested by the computer's users, an operating system must be able to communicate with those users. The portion of an operating system that handles this communication is often called the **user interface**. Older user interfaces, called **shells**, communicated with users through textual messages using a keyboard and monitor screen.

Kernel: In contrast to an operating system's user interface, the internal part of an operating system is called the **Kernel**. An operating system's kernel contains those software components that perform the very basic functions required by the computer installation.

- One such unit is the **file manager**; whose job is to coordinate the use of the machine's mass storage facilities.
- Another component of the kernel consists of a collection of **device drivers**, which are the software units that communicate with the controllers (or at times, directly with peripheral devices) to carry out operations on the peripheral devices attached to the machine. Each device driver is uniquely designed for its particular type of device (such as a printer, disk drive, or monitor).
- Still another component of an operating system's kernel is the **memory manager**, which is charged with the task of coordinating the machine's use of main memory.
- Two additional components within the kernel of an operating system are the **scheduler** and **dispatcher**, which we will study in the next section.

Process of Booting:

We have seen that an operating system provides the software infrastructure required by other software units, but we have not considered how the operating system gets started. This is accomplished through a procedure known as **boot strapping** (often shortened to **booting**) that is performed by a computer each time it is turned on. It is this procedure that transfers the operating system from mass storage (where it is permanently stored) into main memory (which is essentially empty when the machine is first turned on).

Process and its Administration:

Program vs Process:

- Fundamental concepts of modern operating systems are the distinction between a program and the activity of executing a program.
- Static vs dynamic activity.
- Analogy of music sheet.

Process:

- The activity of executing a program under the control of the operating system is known as a process.
- Associated with a process is the current status of the activity, called the process state.

Process state:

- The value of Program counter
- Values in other CPU registers
- Values in associated memory cell.
- Snapshot of the machine at particular time.

The tasks associated with coordinating the execution of processes are handled by **the scheduler and dispatcher** within the operating system's kernel. To keep track of all the processes, the **scheduler** maintains a block of information in main memory called the **process table**. The **dispatcher** is the component of the kernel that oversees the execution of the scheduled processes. The procedure of changing from one process to another is called a **process switch** (or a **context switch**).

A properly implemented flag, as just described, is called a **semaphore**, in reference to the railroad signals used to control access to sections of track.

Deadlock: Another problem that can arise during resource allocation is deadlock, the condition in which two or more processes are blocked from progressing because each is waiting for a resource that is allocated to another.

Attacks from Outside/Inside:

An important task performed by operating systems is to protect the computer's resources from access by unauthorized personnel. Accounts are established by a person known as the **super user** or the **administrator**. This person gains highly privileged access to the operating system by identifying him- or herself as the administrator (usually by name and password) during the **login procedure**.

CPUs for multiprogramming systems are designed to operate in one of two privilege levels; we will call one "**privileged mode**," the other we will call "**non-privileged mode**." When in privileged mode, the CPU is able to execute all the instructions in its machine language. However, when in non-privileged mode, the list of acceptable instructions is limited. The instructions that are available only in privileged mode are called **privileged instructions**.

Introduction to Computer Science – CS101

Lecture 5 – Networking and Internet: Modules (59 to 84)

Network Classification:

A computer network is often classified as being a **personal area network (PAN)**, a local **area network (LAN)**, a **metropolitan area network (MAN)**, or a **wide area network (WAN)**.

A network of the former type is called an **open network**; a network of the latter type is called a **closed network**, or sometimes a **proprietary network**.

Still another way of classifying networks is based on the **topology** of the network, which refers to the pattern in which the machines are connected. Two of the more popular topologies are the **bus**, in which the machines are all connected to a common communication line called a **bus**, and the **star**, in which one machine serves as a central focal point to which all the others are connected.

Protocols: For a network to function reliably, it is important to establish rules by which activities are conducted. Such rules are called **protocols**.

Combining Networks:

Sometimes it is necessary to connect existing networks to form an extended communication system. The simplest of these is the **repeater**, which is little more than a device that passes signals back and forth between the two original buses (usually with some form of amplification) without considering the meaning of the signals.

A **bridge** is similar to, but more complex than, a repeater. Like a repeater, it connects two buses, but it does not necessarily pass all messages across the connection. Instead, it looks at the destination address that accompanies each message and forwards a message across the connection only when that message is destined for a computer on the other side.

A **switch** is essentially a bridge with multiple connections, allowing it to connect several buses rather than just two. The connection between networks to form an internet is handled by devices known as **routers**, which are special purpose computers used for forwarding messages.

The “point” at which one network is linked to an internet is often called a **gateway** because it serves as a passageway between the network and the outside world.

Distributed Systems:

Cluster computing describes a distributed system in which many independent computers work closely together to provide computation or services comparable to a much larger machine. The cost of these individual machines, plus the high-speed network to connect them, can be less than a higher-priced supercomputer, but with higher reliability and lower maintenance costs.

Grid computing refers to distributed systems that are more loosely coupled than clusters but that still work together to accomplish large tasks.

Cloud computing, whereby huge pools of shared computers on the network can be allocated for use by clients as needed, is the latest trend in distributed systems.

Internet Addressing:

Internet needs an internet-wide addressing system that assigns a unique identifying address to each computer in the system. In the Internet these addresses are known as IP addresses.

Internet Applications:

- Network News Transfer Protocol (NNTP)
- File Transfer Protocol (FTP)
- Secure Shell (SSH)
- Hyper Text Transfer Protocol (HTTP)
- VoIP (Voice over Internet Protocol)
- Internet Multimedia Streaming
- Messaging applications

Web implementations: Software packages that allow user to access hypertext on the Internet fall into one of two categories 1) **Browsers** 2) **Web Servers**. Hypertext document are normally transferred between browsers and web servers using a protocol known as the **Hypertext Transfer Protocol (HTTP)**. In order to locate and retrieve document on the Web, each document is given a unique address called a **Uniform Resource Locator (URL)**.

Hypertext Markup Language (HTML):

```
<!DOCTYPE html>
<html>
  <head>
    <title>Introduction to Computer Science – CS101</title>
  </head>
  <body>
    <h1>HTML</h1>
    <p>Hypertext Markup Language (HTML)</p>
  </body>
</html>
```

Layered Approach to Internet Software:

A message typically originates in the application layer. From there it is passed down through the transport and network layers as it is prepared for transmission, and finally it is transmitted by the link layer. The message is received by the link layer at the destination and passed back up the hierarchy until it is delivered to the application layer at the message's destination.

- Application Layer
- Transport Layer
- Network Layer
- Link Layer

Introduction to Computer Science – CS101

Lecture 6 – Algorithm: Modules (85 to 108)

Algorithm: An algorithm is an ordered set of unambiguous, executable steps that defines a terminating process. An algorithm is abstract and distinct from its representation.

Natural channels of communication lead to misunderstandings, sometimes because the terminology used has more than one meaning. Computer science approaches these problems by establishing a well-defined set of building blocks from which algorithm representations can be constructed. Such a building block is called a **primitive**. Each primitive has its own syntax and semantics.

Representation (Pseudocode): A pseudocode must have a consistent, concise notation for representing recurring semantic structures. Depending on whether the year is a leap year, divide the total by 366 or 365, respectively might possess a more creative literary style, we will consistently opt for the straightforward condition:

if (year is leap year): daily total = total / 366

else:

daily total = total / 365

Discovery (The Art of Problem Solving):

1. Phase 1. Understand the problem.
2. Phase 2. Get an idea of how an algorithmic function might solve the problem.
3. Phase 3. Formulate the algorithm and represent it as a program.
4. Phase 4. Evaluate the program for accuracy and for its potential as a tool for solving other problems

Iterative Structures (Sequential Search Algorithm):

In our pseudocode, this process can be represented as:

Select the first entry in the list as TestEntry. while (TargetValue > TestEntry and entries remain):

Select the next entry in the list as TestEntry

Upon terminating this while structure, one of two conditions will be true; either the target value has been found or the target value is not in the list. In either case we can detect a successful search by comparing the test entry to the target value. If they are equal, the search has been successful. Thus, we add the statement

if (TargetValue == TestEntry):

Declare the search a success.

else:

Declare the search a failure.

To the end of our pseudocode routine.

Iterative Structures (Loop Control and Components):

The repetitive use of an instruction or sequence of instructions is an important algorithmic concept. One method of implementing such repetition is the iterative structure known as the **loop**, in which a collection of instructions, called **the body of the loop**, is executed in a repetitive fashion under the direction of some control process. The control of a loop consists of the three activities **initialize**, **test**, and **modify**.

while (condition):

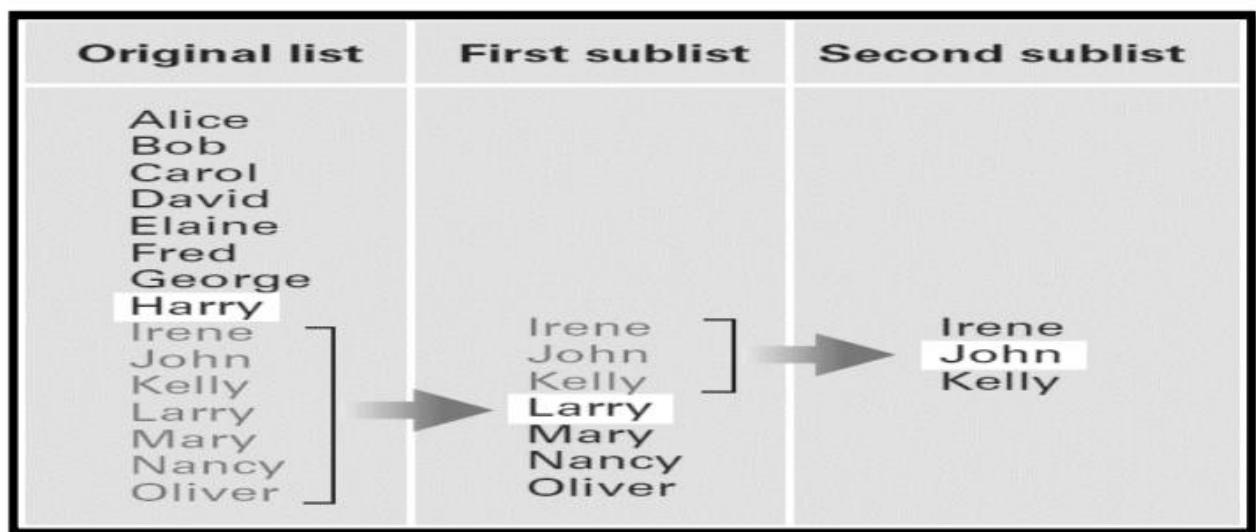
Body

Example of For loop in C++:

```
for(i = 0; i < 10; i++){  
    cout << i ;  
}
```

Iterative Structures (Pretest and Posttest loops): In a more generic context you might hear the while loop structure referred to as a **pretest loop** (since the test for termination is performed before the body is executed) and the repeat loop structure referred to as a **posttest loop** (since the test for termination is performed after the body is executed).

Recursive Structure (The Binary Search Algorithm): We consider the task of searching the list on the left of the figure for the entry **John**. We first consider the middle entry Harry. Since our target belongs after this entry, the search continues by considering the lower half of the original list. The middle of this sub list is found to be Larry. Since our target should precede Larry, we turn our attention to the first half of the current sub list. When we interrogate the middle of that secondary sub list, we find our target John and declare the search a success. In short, our strategy is to successively divide the list in question into smaller segments until the target is found or the search is narrowed to an empty segment.



Introduction to Computer Science – CS101

Lecture 7 – Programming Languages: Modules (109 to 125)

Early Generations-I-II-III:

Once such a mnemonic system was established, programs called **assemblers** were developed to convert mnemonic expressions into machine language instructions. A mnemonic system for representing programs is collectively called an **assembly language**. In fact, assembly languages were so revolutionary that they became known as **second-generation languages**, the **first generation** being the machine languages themselves.

Once this collection of high-level primitives had been identified, a program, called a **translator**, was written that translated programs expressed in these high-level primitives into machine-language programs. Thus, these translation programs were often called **compilers**. An alternative to translators, called **interpreters**, emerged as another means of implementing third generation languages.

Machine Independence: With the development of third-generation languages, the goal of machine independence was largely achieved.

Paradigms:

Imperative Paradigms: In reality, the development of programming languages has not progressed in this manner but has developed along different paths as alternative approaches to the programming process (called **programming paradigms**) have surfaced and been pursued. The **imperative paradigm**, also known as the **procedural paradigm**, represents the traditional approach to the programming process. It is the paradigm on which Python and our pseudocode are based as well as the machine language.

Declarative Paradigms: In contrast to the imperative paradigm is the declarative paradigm, which asks a programmer to describe the problem to be solved rather than an algorithm to be followed.

Functional Paradigm: Another programming paradigm is the functional paradigm. Under this paradigm a program is viewed as an entity that accepts inputs and produces outputs. Mathematicians refer to such entities as **functions**, which is the reason this approach is called the **functional paradigm**.

Object Oriented Paradigm: Another programming paradigm (and the most prominent one in today's software development) is the object-oriented paradigm, which is associated with the programming process called object-oriented programming (OOP). Following this paradigm, a software system is viewed as a collection of units, called objects, each of which is capable of performing the actions that are immediately related to itself as well as requesting actions of other objects. Together, these objects interact to solve the problem at hand.

Variable and Data Types:

High-level programming languages allow locations in main memory to be referenced by descriptive names rather than by numeric addresses. Such a name is known as **variables**. Data type encompasses both the manner in which the data item is encoded and the operations that can be performed on that data.

Integer refers to numeric data consisting of whole numbers, probably stored using two's complement notation. Operations that can be performed on integer data include the traditional arithmetic operations and comparisons of relative size, such as determining whether one value is greater than another.

The type **float** (sometimes called **real**) refers to numeric data that might contain values other than whole numbers, probably stored in floating-point notation. Operations performed on data of type float are similar to those performed on data of type integer.

The type **character** refers to data consisting of symbols, stored using ASCII or Unicode.

The type **Boolean** refers to data items that can take on only the values true or false. The data types that are included as primitives in a programming language, such as int for integer and char for character, are called **primitive data types**.

Data Structure: In addition to data type, variables in a program are often associated with data structure, which is the conceptual shape or arrangement of data. One common data structure is the **Array**, which is a block of elements of the same type such as a **one dimensional** list, a **two dimensional table** with rows and columns, or tables with **higher dimensions**.

Assignment Statement: The most basic imperative statement is the **assignment statement**, which requests that a value be assigned to a variable (or more precisely, stored in the memory area identified by the variable). For example, the statement: **Z = X + Y;**

Control Structures:

Control Structures (if-statement): Control statement alters the execution sequence of the program of all the programming constructs.

```
if (condition)
    Statement A
else
    Statement B
```

Control Structures (Loops): There is another type of control structure known as loop. The loop control structure iterates a set of instructions based on the provided condition.

```
int i=1;
while(i<=5)
{
    cout<<i;
    i=i+1;
}
```

Arithmetic Operators:

- + Addition
- - Subtraction
- * Multiplication
- / Division
- % Modulus

Relational Operators:

- < Less than
- <= Less than or equal to
- > Greater than
- >= Greater than or equal to
- == Equal to
- != Not Equal to

Logical Operators:

Operators	Name of the Operator	Type
&&	AND Operator	Binary
	OR Operator	Binary
!	NOT Operator	Unary

According to names of the Logical Operators, the condition satisfied in following situation and expected outputs are given:

Operator	Output
AND Operator	Output is 1 only when conditions on both sides of Operator become True
OR Operator	Output is 0 only when conditions on both sides of Operator become False
NOT Operator	It gives inverted Output